

Code Explanation

Test Main Function Execution

This test suite is designed to verify the execution flow of the main function in the main module. The test checks the creation of a SparkSession, the instantiation of the AddressChangeProcessor class, and the execution of the process method.

The test code is written using the Pytest framework and utilizes mocking to isolate dependencies.

Overview of the Code

The code under test is the main function in the main module, which is responsible for creating a SparkSession, instantiating the AddressChangeProcessor class, and executing the process method. The test code mocks the SparkSession and AddressChangeProcessor classes to verify their creation and usage.

Step 1: Importing necessary modules and adding the src directory to the path

The test code starts by importing the necessary modules, including Pytest, os, sys, and unittest.mock. It also adds the src directory to the sys.path to enable importing the main module.

```
import pytest
import os
import sys
from unittest.mock import patch, MagicMock
sys.path.append(os.path.join(os.path.dirname(__file__), "../src"))
import main
```

Step 2: Mocking the SparkSession and AddressChangeProcessor classes

The test code uses the @patch decorator to mock the SparkSession and AddressChangeProcessor classes. This allows the test to control the behavior of these classes and verify their usage.

```
@patch("main.SparkSession")
@patch("main.AddressChangeProcessor")
def test_main_execution(mock_processor_class, mock_spark_builder):
```

Step 3: Setting up mock objects for SparkSession and AddressChangeProcessor

The test code creates mock objects for SparkSession and AddressChangeProcessor using the MagicMock class. It sets up the mock SparkSession object to return a mock SparkSession instance when created.

```
mock_spark = MagicMock()
mock_spark_builder.builder.appName.return_value.enableHiveSupport.return_value.getOrCreate.return_value = mock_spark
mock_processor = MagicMock()
mock_processor_class.return_value = mock_processor
```

Step 4: Mocking command line arguments

The test code mocks the command line arguments using the patch decorator to replace sys.argv with a test argument list.

```
test_args = [
    "--process-userid", "TEST_USER",
    "--source-table", "test_source",
    "--target-table", "test_target",
    "--output-path", "/tmp/test_output"
]
with patch("sys.argv", ["main.py"] + test_args):
```

Step 5: Executing the main function and verifying its behavior

The test code executes the main function and verifies its behavior by checking the creation of the SparkSession, the instantiation of the AddressChangeProcessor class, and the execution of the process method.

```
main.main()
mock_spark_builder.builder.appName.assert_called_with("E2E Address C
hange YUYU Processing")
mock_spark_builder.builder.appName.return_value.enableHiveSupport.as
sert_called_once()
mock_processor_class.assert_called_with(mock_spark, "TEST_USER")
mock_processor.process.assert_called_with("test_source", "test_targe
t", "/tmp/test_output")
mock_spark.stop.assert_called_once()
```